

PRACTICAL LAB MANUAL | NATURAL LANGUAGE PROCESSING

Text Preprocessing & Text Representation

Reference Solutions - reviewed, corrected and indexed

spaCy | NLTK | scikit-learn | pandas | NumPy

Eight practical questions (Q1 - Q8), run as Spyder cells

Contents

Click any entry to jump straight to that section. Every page also has a *Back to Contents* link in the footer, and the PDF bookmarks panel mirrors this outline.

Getting Started	4
How to run in Spyder	4
One-time installation	4
Practical map	4
Setup cell	4
Code Review Summary	5
Q1. Tokenization using spaCy	6
Q1 - Input text	6
Q1 (a) Print every token with its index	6
Q1 (b) Token attribute table	6
Q1 (c) Counts	6
Q1 (d) Sentence segmentation	7
Q1 (e) spaCy tokenizer vs Python split()	7
Q2. The spaCy Pipeline	8
Q2 - Input reviews	8
Q2 (a) Inspect the pipeline	8
Q2 (b) Speed comparison: full pipeline vs only what is needed	8
Q2 (c) Custom component: word count and estimated reading time	9
Q2 (d) Batch processing with nlp.pipe()	9
Q3. Text Normalization : Stop Words, Stemming & Lemmatization	10
Q3 - Imports and input data	10
Q3 (A) Stop words: count, remove, and customise	10
Q3 (B) Stemming vs Lemmatization	11
Q4. Part-of-Speech (POS) Tagging	12
Q4 - Input text	12
Q4 (a) POS table with explanation	12
Q4 (b) POS frequency	12
Q4 (c) Nouns, verbs (as lemmas) and adjectives	12
Q4 (d) Adjective -> noun pairs using the dependency tree	12
Q4 (e) Noun chunks	13
Q4 (f) Same word, different POS depending on context	13

Q5. Named Entity Recognition (NER)	14
Q5 - Input text	14
Q5 (a) Entities with label and explanation	14
Q5 (b) Count of entities per label	14
Q5 (c) Group entity texts by type for PERSON, ORG, GPE	14
Q5 (d) Custom entities with EntityRuler (on a separate copy of the model)	14
Q5 (e) Visualise with displaCy and save to an HTML file	15
Q6. Word Embeddings	16
Q6 - Load the large model	16
Q6 (a) Inspect a vector	16
Q6 (b) Similarity matrix	16
Q6 (c) Nearest neighbours (helper functions rank words by cosine similarity)	16
Q6 (d) Word analogy: king - man + woman ~ queen	17
Q6 (e) Sentence similarity and how a doc vector is built	17
Q7. Bag of Words & Bag of N-grams	18
Q7 - Imports and corpus	18
Q7 (a) Bag of Words from scratch	18
Q7 (b) Same with CountVectorizer and verify	18
Q7 (c) Binary BoW (presence / absence)	19
Q7 (d) Bag of N-grams and vocabulary growth	19
Q7 (e) Why n-grams matter: word order	19
Q7 (f) Unseen words in a new sentence	19
Q8. TF-IDF and Document Search	20
Q8 - Imports and corpus	20
Q8 (a) Preprocess with spaCy: lowercase lemma, no stop words / punctuation	20
Q8 (b) TF-IDF from scratch (same formula as sklearn's default)	20
Q8 (c) Top 3 keywords per document	21
Q8 (d) Document-to-document cosine similarity	21
Q8 (e) Simple search engine	21
Q8 (f) Compare with plain Bag of Words for the same query	21

Getting Started

This document contains the eight practical questions on text preprocessing and text representation together with cleaned-up, tested solution code. Each question is one Spyder cell (marked with # %%), split here into lettered parts so you can find and read every step.

How to run in Spyder

- Run the **Setup** cell first (it loads spaCy and the small English model).
- Run any question cell with **Ctrl + Enter**; output appears in the IPython console.
- No dataset is needed: all text is defined inside the code.
- Cells can be re-run safely; Q2 and Q3 restore the model state at the end.

One-time installation

Run these in the Anaconda Prompt:

```
pip install spacy nltk scikit-learn pandas numpy
python -m spacy download en_core_web_sm
python -m spacy download en_core_web_lg      # needed only for Q6 (~400 MB)
```

Practical map

Q	Topic	Main tools	Model needed
Q1	Tokenization	spaCy tokenizer, pandas	en_core_web_sm
Q2	The spaCy pipeline	nlp.pipe, custom component	en_core_web_sm
Q3	Stop words, stemming, lemmatization	spaCy, NLTK stemmers	en_core_web_sm
Q4	POS tagging	spaCy tagger, parser	en_core_web_sm
Q5	Named entity recognition	spaCy NER, EntityRuler, displaCy	en_core_web_sm
Q6	Word embeddings	spaCy vectors, NumPy	en_core_web_lg
Q7	Bag of Words and n-grams	CountVectorizer	en_core_web_sm
Q8	TF-IDF and document search	TfidfVectorizer, cosine similarity	en_core_web_sm

Setup cell

Run this cell before any question. It imports the shared libraries, configures pandas output and loads the small model.

```
import time
from collections import Counter

import numpy as np
import pandas as pd
import spacy
from spacy import displacy
from spacy.language import Language
from spacy.tokens import Doc

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 30)
pd.set_option("display.max_rows", 100)

nlp = spacy.load("en_core_web_sm")
print("spaCy version :", spacy.__version__)
print("Model loaded  :", nlp.meta["name"])
```

Code Review Summary

The original solution was reviewed line by line and then executed end to end. The logic of the eight questions was sound, but the items below were wrong or misleading and have been fixed in this document.

#	Where	Problem found	Fix applied
1	File header	The banner block ends with <code>"""</code> but never opens with one, so Python stops with a SyntaxError (unterminated string) before any cell runs.	Added the opening <code>"""</code> so the banner is a proper module docstring.
2	Q1 (e)	The list printed under <code>"split() keeps these together"</code> actually holds the words that spaCy <i>breaks apart</i> (<i>Rs.</i> , <i>Crossword</i> ., <i>Isn't...</i>). The label said the opposite.	Label changed to <code>"split() words that spaCy breaks apart"</code> .
3	Q2 (b)	The first timed run also pays one-off start-up cost, so the reported speed-up was overstated.	Added an untimed warm-up call before timing.
4	Q3 (B)	The explanation named <i>better</i> although the sentence contains only <i>best</i> . The variable <code>not_real</code> never tested whether a word was real. Isolated irregular words (<i>geese</i> , <i>easily</i>) are not always lemmatized.	Text corrected, variable renamed <code>differs</code> , and a comment explains the isolated-word limitation.
5	Q6 (b)/(c)	Helper functions sat between parts (b) and (c), so part (b) looked incomplete.	Moved the helpers into part (c); behaviour unchanged.
6	Q8 (f)	The closing claim that TF-IDF "raises rare terms" was printed without any evidence in the output.	Now prints the D6/D5 score ratio (2.00x for BoW vs 2.49x for TF-IDF).
7	Whole file	One-space indentation: valid, but non-standard and hard to read.	Re-indented to 4 spaces (PEP 8).

Verification

All cells (Setup, Q1 to Q8) were run in sequence on Python 3.12, spaCy 3.8 (`en_core_web_sm` and `en_core_web_lg 3.8.0`), NLTK 3.10 and scikit-learn 1.8 with no errors. Checks that came out as expected: manual Bag of Words equals `CountVectorizer`; manual TF-IDF equals `TfidfVectorizer`; the document vector equals the mean of token vectors; *not* is kept and *movie* removed after stop-word customisation.

Note on numbers

Timing results (Q2) and similarity scores (Q6) can differ slightly between machines and library versions. Model-dependent output such as POS tags and entities may also vary with the spaCy model version.

Q1. Tokenization using spaCy

Tokenization is the first step of almost every NLP pipeline: raw text is cut into tokens (words, numbers, punctuation). spaCy uses a rule-based tokenizer with exceptions for abbreviations, contractions, e-mail addresses and URLs. This practical prints the tokens, inspects their attributes, counts them by type, segments the text into sentences and compares the result with Python's `str.split()`.

Key takeaway

`str.split()` only cuts on whitespace, so punctuation stays glued to words (Crossword, , amazing?). spaCy separates punctuation, splits contractions such as *Isn't* into *Is* + *n't*, and still keeps e-mail addresses, URLs and *Dr.* intact.

Q1 - Input text

```
q1_text = ("Dr. Sharma paid Rs. 1,250.50 for 3 books at Crossword, Mumbai on "
           "15-Aug-2024! Email her at sharma.r@gmail.com or visit "
           "https://www.example.com for details. Isn't it amazing? "
           "She didn't expect a 20% discount.")
doc = nlp(q1_text)
```

Q1 (a) Print every token with its index

```
# (a) Print every token with its index
print("(a) Tokens")
for token in doc:
    print(f"{token.i:>3} | {token.text}")
```

Q1 (b) Token attribute table

```
# (b) Token attribute table
rows = []
for token in doc:
    rows.append({
        "token": token.text,
        "char_idx": token.idx,
        "is_alpha": token.is_alpha,
        "is_punct": token.is_punct,
        "like_num": token.like_num,
        "like_email": token.like_email,
        "like_url": token.like_url,
        "is_currency": token.is_currency,
        "shape": token.shape_,
    })
q1_df = pd.DataFrame(rows)
print("\n(b) Token attributes\n", q1_df)
```

Q1 (c) Counts

```
# (c) Counts
total_tokens = len(doc)
word_tokens = sum(1 for t in doc if t.is_alpha)
punct_tokens = sum(1 for t in doc if t.is_punct)
num_tokens = sum(1 for t in doc if t.like_num)
print(f"\n(c) Total tokens : {total_tokens}")
print(f"    Word tokens : {word_tokens}")
print(f"    Punctuation : {punct_tokens}")
print(f"    Numbers      : {num_tokens}")
```

Q1 (d) Sentence segmentation

```
# (d) Sentence segmentation
print("\n(d) Sentences")
for i, sent in enumerate(doc.sents, start=1):
    print(f"    S{i}: {sent.text}")
```

Q1 (e) spaCy tokenizer vs Python split()

```
# (e) spaCy tokenizer vs Python split()
split_tokens = q1_text.split()
spacy_tokens = [t.text for t in doc]
print(f"\n(e) str.split() tokens : {len(split_tokens)}")
print(f"    spaCy tokens          : {len(spacy_tokens)}")
print("    split() words that spaCy breaks apart :",
      [w for w in split_tokens if w not in spacy_tokens])
print("    Observation: spaCy separates punctuation and splits contractions "
      "(Isn't -> Is + n't), but keeps emails, URLs and 'Dr.' as single tokens.")
```

Q2. The spaCy Pipeline

A spaCy pipeline is a sequence of components (tok2vec, tagger, parser, attribute_ruler, lemmatizer, ner) that add annotations to a Doc after the tokenizer has run. This practical inspects the pipeline, measures the speed gained by disabling unused components, adds a custom component that stores word count and reading time, and processes texts in batches with `nlp.pipe()`.

Key takeaway

Only run the components you need: disabling *parser* and *ner* roughly halves processing time here. Custom components and extension attributes (`doc._.name`) let you plug your own logic into the pipeline.

Q2 - Input reviews

```
q2_reviews = [
    "The delivery was quick and the packaging was excellent.",
    "Worst customer service I have ever experienced.",
    "Battery life of this phone is amazing, lasts two full days.",
    "The laptop heats up quickly and the fan is very noisy.",
    "Good value for money, I would recommend it to my friends.",
]
```

Q2 (a) Inspect the pipeline

```
# (a) Inspect the pipeline
print("(a) Pipeline components :", nlp.pipe_names)
for name, component in nlp.pipeline:
    print(f"    {name:<12} -> {type(component).__name__}")
print("    Note: the tokenizer always runs first and is not listed in pipe_names.")
```

Q2 (b) Speed comparison: full pipeline vs only what is needed

```
# (b) Speed comparison: full pipeline vs only what is needed
big_corpus = q2_reviews * 200          # 1000 short documents

list(nlp.pipe(q2_reviews))             # warm-up run (loads caches, not timed)

start = time.perf_counter()
full_docs = list(nlp.pipe(big_corpus))
full_time = time.perf_counter() - start

with nlp.select_pipes(disable=["parser", "ner"]):
    start = time.perf_counter()
    light_docs = list(nlp.pipe(big_corpus))
    light_time = time.perf_counter() - start

print(f"\n(b) Full pipeline           : {full_time:.3f} s")
print(f"    Without parser and ner : {light_time:.3f} s")
print(f"    Speed-up                 : {full_time / light_time:.2f}x")
print(f"    Entities available after disabling ner? ",
      light_docs[0].has_annotation("ENT_IOB"))
```

Q2 (c) Custom component: word count and estimated reading time

```
# (c) Custom component: word count and estimated reading time
if not Doc.has_extension("word_count"):
    Doc.set_extension("word_count", default=0)
if not Doc.has_extension("reading_time_sec"):
    Doc.set_extension("reading_time_sec", default=0.0)

@Language.component("text_stats")
def text_stats(doc):
    words = [t for t in doc if t.is_alpha]
    doc._word_count = len(words)
    doc._reading_time_sec = round(len(words) / 200 * 60, 2)  # 200 words/min
    return doc

if "text_stats" not in nlp.pipe_names:
    nlp.add_pipe("text_stats", last=True)
print("\n(c) Pipeline after adding custom component :", nlp.pipe_names)
```

Q2 (d) Batch processing with nlp.pipe()

```
# (d) Batch processing with nlp.pipe()
print("\n(d) Batch results")
for review_doc in nlp.pipe(q2_reviews):
    print(f"    words={review_doc._word_count:>2} | "
          f"read={review_doc._reading_time_sec:>4}s | {review_doc.text}")

# Clean up so later questions use the standard pipeline
nlp.remove_pipe("text_stats")
```

Q3. Text Normalization : Stop Words, Stemming & Lemmatization

Text normalization reduces the variety of word forms. Stop-word removal drops very frequent function words; stemming chops suffixes with rules; lemmatization maps a word to its dictionary form using its part of speech. This practical measures the effect of stop-word removal, customises the stop-word list (keeping *not* for sentiment tasks) and compares the Porter, Snowball and Lancaster stemmers with spaCy's lemmatizer.

Key takeaway

Stems such as *easili* or *happi* are not real words, and Lancaster is the most aggressive stemmer. Lemmas are real words but need context: spaCy uses the POS tag to turn *ate* into *eat* and *best* into *good*. Never blindly remove *not* in sentiment analysis.

Q3 - Imports and input data

```
from nltk.stem import LancasterStemmer, PorterStemmer, SnowballStemmer

q3_words = ["running", "runs", "ran", "runner", "easily", "fairly", "studies",
            "studying", "better", "wolves", "geese", "happiness",
            "organization", "organizing", "generously"]
q3_text = ("The movie was not good at all. The actors were trying very hard but "
           "the story was boring and the songs were too long. I would not "
           "watch this movie again, although the music was quite nice.")
```

Q3 (A) Stop words: count, remove, and customise

```
# (A) Stop words: count, remove, and customise
stop_words = nlp.Defaults.stop_words
print("(A1) Number of default stop words :", len(stop_words))
print("    First 20 (sorted) :", sorted(stop_words)[:20])

doc = nlp(q3_text)
tokens_all = [t.text.lower() for t in doc if not t.is_punct]
tokens_clean = [t.text.lower() for t in doc if not t.is_stop and not t.is_punct]
reduction = 100 * (1 - len(tokens_clean) / len(tokens_all))
print(f"\n(A2) Tokens before : {len(tokens_all)}")
print(f"    Tokens after  : {len(tokens_clean)} ({reduction:.1f}% reduction)")
print("    Clean tokens  :", tokens_clean)
print("\n(A3) Top 5 words before :", Counter(tokens_all).most_common(5))
print("    Top 5 words after  :", Counter(tokens_clean).most_common(5))

# Customising stop words: keep 'not' (important for sentiment), add 'movie'
nlp.Defaults.stop_words.discard("not")
nlp.vocab["not"].is_stop = False
nlp.Defaults.stop_words.add("movie")
nlp.vocab["movie"].is_stop = True

doc_custom = nlp(q3_text)
tokens_custom = [t.text.lower() for t in doc_custom
                 if not t.is_stop and not t.is_punct]
print("\n(A4) With custom stop words :", tokens_custom)
print("    'not' kept? ", "not" in tokens_custom,
      "| 'movie' removed? ", "movie" not in tokens_custom)

# Restore defaults so later questions are not affected
nlp.Defaults.stop_words.add("not")
nlp.vocab["not"].is_stop = True
nlp.Defaults.stop_words.discard("movie")
nlp.vocab["movie"].is_stop = False
```

Q3 (B) Stemming vs Lemmatization

```
# (B) Stemming vs Lemmatization
porter = PorterStemmer()
snowball = SnowballStemmer("english")
lancaster = LancasterStemmer()

rows = []
for w in q3_words:
    rows.append({
        "word": w,
        "porter": porter.stem(w),
        "snowball": snowball.stem(w),
        "lancaster": lancaster.stem(w),
        "spacy_lemma": nlp(w)[0].lemma_,
    })
q3_df = pd.DataFrame(rows)
print("\n(B1) Stemmers vs spaCy lemmatizer\n", q3_df)

# Stems are often not real words ('easili', 'happi'); lemmas are dictionary
# words. Note: spaCy's rule-based lemmatizer works best inside a sentence, so
# isolated irregular words ('geese', 'easily') may come back unchanged.
differs = q3_df[q3_df["porter"] != q3_df["spacy_lemma"]]["word"].tolist()
print("\n(B2) Words where Porter stem differs from the lemma :", differs)

# Lemmatization depends on context (POS)
sentence = "The striped bats were hanging on their feet and ate the best fishes."
print("\n(B3) Context-aware lemmatization")
for t in nlp(sentence):
    if not t.is_punct:
        print(f"      {t.text:<8} POS={t.pos_:<5} lemma={t.lemma_:<8} "
              f"porter={porter.stem(t.text)}")
print("      'best' -> 'good', 'were' -> 'be' and 'feet' -> 'foot' work because "
      "the lemmatizer uses the POS tag. Stemmers just chop suffixes.")
```

Q4. Part-of-Speech (POS) Tagging

POS tagging labels each token with its grammatical role (noun, verb, adjective, ...). spaCy gives a coarse universal tag (pos_) and a fine-grained Penn Treebank tag (tag_). This practical builds a POS table, counts tags, extracts nouns, verbs and adjectives, finds adjective-noun pairs through the dependency tree, lists noun chunks and shows how one word can receive different tags in different contexts.

Key takeaway

POS depends on context, not on the word alone: *book* is a VERB in *I will book a flight* and a NOUN in *a good book*. Dependency labels such as amod link modifiers to the words they describe.

Q4 - Input text

```
q4_text = ("The young engineer quickly wrote clean Python code for the new "
           "banking app. Her manager was very happy and gave her a small "
           "bonus. The team will release the beautiful dashboard next week.")
doc = nlp(q4_text)
```

Q4 (a) POS table with explanation

```
# (a) POS table with explanation
rows = [{"token": t.text,
        "pos": t.pos_,
        "tag": t.tag_,
        "explanation": spacy.explain(t.tag_)} for t in doc]
print("(a) POS tags\n", pd.DataFrame(rows))
```

Q4 (b) POS frequency

```
# (b) POS frequency
pos_counts = Counter(t.pos_ for t in doc if not t.is_punct)
print("\n(b) POS frequency :", pos_counts.most_common())
```

Q4 (c) Nouns, verbs (as lemmas) and adjectives

```
# (c) Nouns, verbs (as lemmas) and adjectives
nouns = [t.text for t in doc if t.pos_ in ("NOUN", "PROPN")]
verbs = [t.lemma_ for t in doc if t.pos_ == "VERB"]
adjectives = [t.text for t in doc if t.pos_ == "ADJ"]
print("\n(c) Nouns      :", nouns)
print("      Verbs      :", verbs)
print("      Adjectives   :", adjectives)
```

Q4 (d) Adjective -> noun pairs using the dependency tree

```
# (d) Adjective -> noun pairs using the dependency tree
pairs = [(t.text, t.head.text) for t in doc
         if t.pos_ == "ADJ" and t.dep_ == "amod"]
print("\n(d) Adjective-noun pairs :", pairs)
```

Q4 (e) Noun chunks

```
# (e) Noun chunks
print("\n(e) Noun chunks :", [chunk.text for chunk in doc.noun_chunks])
```

Q4 (f) Same word, different POS depending on context

```
# (f) Same word, different POS depending on context
print("\n(f) Ambiguity")
for s in ["I will book a flight to Delhi.", "I am reading a good book."]:
    for t in nlp(s):
        if t.text == "book":
            print(f"    '{s}' -> book = {t.pos_} ({spacy.explain(t.pos_)})")
```

Q5. Named Entity Recognition (NER)

NER finds real-world objects in text (people, organisations, places, money, dates) and assigns each a label. This practical lists entities with explanations, counts them per label, groups them by type, adds custom entities with an EntityRuler and exports a displaCy visualisation to an HTML file.

Key takeaway

The statistical model recognises general entities out of the box. A rule-based EntityRuler placed *before* ner adds domain labels (TECH, COURSE) that the default model does not know. The file q5_entities.html is written to the working directory.

Q5 - Input text

```
q5_text = ("Sundar Pichai, the CEO of Google, visited Bengaluru on Monday to "  
          "meet Mukesh Ambani of Reliance Industries. Google announced an "  
          "investment of $10 billion in India over the next 5 years. "  
          "The meeting took place at the Taj West End hotel and was "  
          "attended by over 200 people from Infosys, Wipro and TCS.")  
doc = nlp(q5_text)
```

Q5 (a) Entities with label and explanation

```
# (a) Entities with label and explanation  
rows = [{"entity": ent.text,  
        "label": ent.label_,  
        "meaning": spacy.explain(ent.label_),  
        "start_char": ent.start_char,  
        "end_char": ent.end_char} for ent in doc.ents]  
q5_df = pd.DataFrame(rows)  
print("(a) Entities\n", q5_df)
```

Q5 (b) Count of entities per label

```
# (b) Count of entities per label  
print("\n(b) Entities per label\n", q5_df["label"].value_counts())
```

Q5 (c) Group entity texts by type for PERSON, ORG, GPE

```
# (c) Group entity texts by type for PERSON, ORG, GPE  
grouped = {}  
for ent in doc.ents:  
    if ent.label_ in ("PERSON", "ORG", "GPE"):  
        grouped.setdefault(ent.label_, set()).add(ent.text)  
print("\n(c) Grouped entities :", grouped)
```

Q5 (d) Custom entities with EntityRuler (on a separate copy of the model)

```
# (d) Custom entities with EntityRuler (on a separate copy of the model)  
nlp_ruler = spacy.load("en_core_web_sm")  
ruler = nlp_ruler.add_pipe("entity_ruler", before="ner")  
ruler.add_patterns([  
    {"label": "TECH", "pattern": "Python"},  
    {"label": "TECH", "pattern": "spaCy"},  
    {"label": "TECH", "pattern": [{"LOWER": "spyder"}, {"LOWER": "ide"}]},  
    {"label": "COURSE", "pattern": [{"LOWER": "natural"},  
                                   {"LOWER": "language"},  
                                   {"LOWER": "processing"}]},  
])  
q5_custom = ("Students of MIT-WPU Pune use Python, spaCy and Spyder IDE in the "  
            "Natural Language Processing course taught by Prof. Kulkarni.")  
print("\n(d) Default model :",  
      [(e.text, e.label_) for e in nlp(q5_custom).ents])
```

```
print("    With ruler    :",  
      [(e.text, e.label_) for e in nlp_ruler(q5_custom).ents])
```

Q5 (e) Visualise with displaCy and save to an HTML file

```
# (e) Visualise with displaCy and save to an HTML file  
html = displacy.render(doc, style="ent", page=True, jupyter=False)  
with open("q5_entities.html", "w", encoding="utf-8") as f:  
    f.write(html)  
print("\n(e) Saved 'q5_entities.html' in the working directory. "  
      "Open it in a browser.")
```

Q6. Word Embeddings

Word embeddings represent words as dense vectors so that words with similar meanings lie close together. This practical needs `en_core_web_lg` because the small model has no real word vectors. It inspects vectors, builds a similarity matrix, finds nearest neighbours, solves the analogy *king - man + woman* and shows that a document vector is the average of its token vectors.

Key takeaway

Cosine similarity captures meaning: *doctor* is close to *surgeon* and *nurse*, and *king - man + woman* lands nearest to *queen*. Averaging word vectors gives a simple but useful sentence representation, although it ignores word order.

Q6 - Load the large model

```
nlp_lg = spacy.load("en_core_web_lg")
```

Q6 (a) Inspect a vector

```
# (a) Inspect a vector
king = nlp_lg.vocab["king"]
print("(a) Vector dimension of 'king' :", king.vector.shape)
print("    First 5 values :", np.round(king.vector[:5], 3))
for w in ["king", "mumbai", "xyzqwe"]:
    lex = nlp_lg.vocab[w]
    print(f"    {w:<8} has_vector={lex.has_vector} is_oov={lex.is_oov}")
```

Q6 (b) Similarity matrix

```
# (b) Similarity matrix
q6_words = ["king", "queen", "man", "woman", "apple", "mango", "car", "bus"]
tokens = [nlp_lg(w)[0] for w in q6_words]
sim_matrix = pd.DataFrame(
    [[round(t1.similarity(t2), 2) for t2 in tokens] for t1 in tokens],
    index=q6_words, columns=q6_words)
print("\n(b) Similarity matrix\n", sim_matrix)
```

Q6 (c) Nearest neighbours (helper functions rank words by cosine similarity)

```
# (c) Nearest neighbours (helper functions rank words by cosine similarity)
candidates = ["doctor", "nurse", "hospital", "patient", "medicine", "surgeon",
              "cricket", "football", "batsman", "stadium", "tennis", "match",
              "python", "java", "code", "programming", "software", "snake",
              "king", "queen", "prince", "princess", "man", "woman", "boy",
              "girl", "apple", "mango", "banana", "car", "bus", "train"]

def cosine(v1, v2):
    return float(np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2)))

def most_similar(vector, n=5, exclude=()):
    scores = [(w, round(cosine(vector, nlp_lg.vocab[w].vector), 3))
              for w in candidates if w not in exclude]
    return sorted(scores, key=lambda x: x[1], reverse=True)[:n]

for w in ["doctor", "cricket", "python"]:
    print(f"\n(c) Most similar to '{w}':",
          most_similar(nlp_lg.vocab[w].vector, exclude=[w]))
```

Q6 (d) Word analogy: king - man + woman ~ queen

```
# (d) Word analogy: king - man + woman ~ queen
analogy = (nlp_lg.vocab["king"].vector - nlp_lg.vocab["man"].vector
           + nlp_lg.vocab["woman"].vector)
print("\n(d) king - man + woman ~",
      most_similar(analogy, n=3, exclude=["king", "man", "woman"]))
```

Q6 (e) Sentence similarity and how a doc vector is built

```
# (e) Sentence similarity and how a doc vector is built
s1 = nlp_lg("I love eating spicy Indian food.")
s2 = nlp_lg("Delicious curry and biryani are my favourite meals.")
s3 = nlp_lg("The stock market crashed badly today.")
print(f"\n(e) s1 vs s2 : {s1.similarity(s2):.3f}")
print(f"    s1 vs s3 : {s1.similarity(s3):.3f}")
manual_avg = np.mean([t.vector for t in s1], axis=0)
print("    Doc vector == average of token vectors ? ",
      np.allclose(manual_avg, s1.vector))
```

Q7. Bag of Words & Bag of N-grams

Bag of Words (BoW) represents a document as the counts of the words it contains, ignoring order. A bag of n-grams also counts sequences of n words, which recovers some local word order. This practical builds BoW by hand, verifies it against `CountVectorizer`, creates binary features, studies vocabulary growth for several n-gram ranges, shows why order matters and how unseen words are handled.

Key takeaway

good, not bad and *bad, not good* have identical unigram vectors but different bigram vectors. Words that were not in the training vocabulary are silently ignored at transform time.

Q7 - Imports and corpus

```
from sklearn.feature_extraction.text import CountVectorizer

q7_corpus = [
    "The food was good and the service was good.",
    "The food was bad and the service was slow.",
    "Service was quick but the food was cold.",
    "Good food, good service, good price.",
]
```

Q7 (a) Bag of Words from scratch

```
# (a) Bag of Words from scratch
def simple_tokens(text):
    return [t.text.lower() for t in nlp(text) if t.is_alpha]

tokenised = [simple_tokens(d) for d in q7_corpus]
vocab = sorted(set(w for doc_tokens in tokenised for w in doc_tokens))
bow_manual = [[doc_tokens.count(w) for w in vocab] for doc_tokens in tokenised]
bow_manual_df = pd.DataFrame(bow_manual, columns=vocab,
                             index=[f"D{i+1}" for i in range(len(q7_corpus))])
print("(a) Manual Bag of Words\n", bow_manual_df)
```

Q7 (b) Same with CountVectorizer and verify

```
# (b) Same with CountVectorizer and verify
cv = CountVectorizer()
X = cv.fit_transform(q7_corpus)
bow_sklearn_df = pd.DataFrame(X.toarray(), columns=cv.get_feature_names_out(),
                              index=bow_manual_df.index)
print("\n(b) CountVectorizer BoW\n", bow_sklearn_df)
print("    Manual == sklearn ? ", bow_manual_df.equals(bow_sklearn_df))
print(f"    Matrix shape {X.shape}, sparsity = "
      f"{100 * (1 - X.nnz / np.prod(X.shape)):.1f}% zeros")
```

Q7 (c) Binary BoW (presence / absence)

```
# (c) Binary BoW (presence / absence)
cv_bin = CountVectorizer(binary=True)
print("\n(c) Binary BoW\n",
      pd.DataFrame(cv_bin.fit_transform(q7_corpus).toarray(),
                  columns=cv_bin.get_feature_names_out(),
                  index=bow_manual_df.index))
```

Q7 (d) Bag of N-grams and vocabulary growth

```
# (d) Bag of N-grams and vocabulary growth
for ngram in [(1, 1), (2, 2), (1, 2), (1, 3)]:
    cv_n = CountVectorizer(ngram_range=ngram)
    cv_n.fit(q7_corpus)
    print(f"\n(d) ngram_range={ngram} -> vocabulary size "
          f"{len(cv_n.vocabulary_)}")

cv_bi = CountVectorizer(ngram_range=(2, 2))
bi_df = pd.DataFrame(cv_bi.fit_transform(q7_corpus).toarray(),
                    columns=cv_bi.get_feature_names_out(),
                    index=bow_manual_df.index)
print("    Bigram matrix\n", bi_df)
```

Q7 (e) Why n-grams matter: word order

```
# (e) Why n-grams matter: word order
pair = ["The food was good, not bad.", "The food was bad, not good."]
uni = CountVectorizer().fit(pair)
bi = CountVectorizer(ngram_range=(1, 2)).fit(pair)
u = uni.transform(pair).toarray()
b = bi.transform(pair).toarray()
print("\n(e) Unigram vectors identical? ", np.array_equal(u[0], u[1]))
print("    Uni+bigram vectors identical?", np.array_equal(b[0], b[1]))
print("    Bag of Words ignores word order, so opposite sentences look the "
      "same. Bigrams like 'not bad' vs 'not good' capture the difference.")
```

Q7 (f) Unseen words in a new sentence

```
# (f) Unseen words in a new sentence
new_doc = ["The pizza was good but the delivery was late."]
vec = cv.transform(new_doc).toarray()[0]
known = {w: int(c) for w, c in zip(cv.get_feature_names_out(), vec) if c > 0}
unknown = [w for w in simple_tokens(new_doc[0]) if w not in cv.vocabulary_]
print("\n(f) Counted words :", known)
print("    Ignored (OOV) :", unknown)
```

Q8. TF-IDF and Document Search

TF-IDF weights a term by how often it appears in a document (TF) and how rare it is across the corpus (IDF), so distinctive words score higher than common ones. This practical preprocesses text with spaCy, implements TF-IDF from scratch and checks it against `TfidfVectorizer`, extracts keywords, computes document similarity, builds a small search engine and compares it with plain BoW.

Key takeaway

Rows are L2-normalised, so cosine similarity is a dot product. Ranking by TF-IDF cosine similarity puts the most relevant document first, and rare words such as *food* contribute more than common ones such as *indian*.

Q8 - Imports and corpus

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

q8_corpus = [
    "India won the cricket match against Australia by six wickets.",
    "Virat Kohli scored a brilliant century in the cricket test match.",
    "The new smartphone has a powerful processor and a great camera.",
    "Apple launched a new laptop with a faster processor and longer battery.",
    "Paneer butter masala and garlic naan are popular Indian dishes.",
    "This restaurant serves delicious Indian food like biryani and dosa.",
]
doc_ids = [f"D{i+1}" for i in range(len(q8_corpus))]
```

Q8 (a) Preprocess with spaCy: lowercase lemma, no stop words / punctuation

```
# (a) Preprocess with spaCy: lowercase lemma, no stop words / punctuation
def preprocess(text):
    return " ".join(t.lemma_.lower() for t in nlp(text)
                    if t.is_alpha and not t.is_stop)

clean_corpus = [preprocess(d) for d in q8_corpus]
print("(a) Preprocessed corpus")
for i, c in zip(doc_ids, clean_corpus):
    print(f"    {i}: {c}")
```

Q8 (b) TF-IDF from scratch (same formula as sklearn's default)

```
# (b) TF-IDF from scratch (same formula as sklearn's default)
# tf = raw count of term in document
# idf = ln((1 + N) / (1 + df)) + 1
# tfidf = tf * idf, then each row is L2-normalised
cv = CountVectorizer()
tf = cv.fit_transform(clean_corpus).toarray()
terms = cv.get_feature_names_out()
N = tf.shape[0]
df = (tf > 0).sum(axis=0)
idf = np.log((1 + N) / (1 + df)) + 1
tfidf_manual = tf * idf
tfidf_manual = tfidf_manual / np.linalg.norm(tfidf_manual, axis=1, keepdims=True)

tfidf_vec = TfidfVectorizer()
tfidf_sklearn = tfidf_vec.fit_transform(clean_corpus).toarray()
print("\n(b) Manual TF-IDF == sklearn TF-IDF ? ",
      np.allclose(tfidf_manual, tfidf_sklearn))

idf_df = pd.DataFrame({"term": terms, "df": df, "idf": np.round(idf, 3)})
print("    Lowest IDF (common) terms\n",
      idf_df.sort_values("idf").head(5).to_string(index=False))

tfidf_df = pd.DataFrame(np.round(tfidf_sklearn, 3),
```

```

        columns=tfidf_vec.get_feature_names_out(),
        index=doc_ids)
print("\n    TF-IDF matrix\n", tfidf_df)

```

Q8 (c) Top 3 keywords per document

```

# (c) Top 3 keywords per document
print("\n(c) Top 3 keywords per document")
for i in doc_ids:
    top = tfidf_df.loc[i].sort_values(ascending=False).head(3)
    print(f"    {i}: {list(top.index)}")

```

Q8 (d) Document-to-document cosine similarity

```

# (d) Document-to-document cosine similarity
sim = pd.DataFrame(np.round(cosine_similarity(tfidf_sklearn), 2),
                  index=doc_ids, columns=doc_ids)
print("\n(d) Cosine similarity between documents\n", sim)
sim_values = sim.to_numpy(copy=True)
np.fill_diagonal(sim_values, 0) # ignore self-similarity
r, c = np.unravel_index(sim_values.argmax(), sim_values.shape)
print(f"    Most similar pair : {doc_ids[r]} & {doc_ids[c]}")

```

Q8 (e) Simple search engine

```

# (e) Simple search engine
def search(query, top_k=3):
    q_vec = tfidf_vec.transform([preprocess(query)])
    scores = cosine_similarity(q_vec, tfidf_sklearn)[0]
    ranked = np.argsort(scores)[::-1][:top_k]
    return [(doc_ids[r], round(float(scores[r]), 3), q8_corpus[r])
            for r in ranked]

for query in ["cricket match score", "laptop with good battery",
             "tasty Indian dishes"]:
    print(f"\n(e) Query: '{query}'")
    for did, score, text in search(query):
        print(f"    {did} score={score:.2f} {text}")

```

Q8 (f) Compare with plain Bag of Words for the same query

```

# (f) Compare with plain Bag of Words for the same query
bow_scores = cosine_similarity(cv.transform([preprocess("Indian food")]),
                              tf)[0]
tfidf_scores = cosine_similarity(
    tfidf_vec.transform([preprocess("Indian food")]), tfidf_sklearn)[0]
print("\n(f) Query 'Indian food' BoW vs TF-IDF scores")
print(pd.DataFrame({"Bow": np.round(bow_scores, 3),
                   "TF-IDF": np.round(tfidf_scores, 3)}, index=doc_ids))
i5, i6 = doc_ids.index("D5"), doc_ids.index("D6")
print(f"    D6 / D5 score ratio -> BoW: {bow_scores[i6] / bow_scores[i5]:.2f}x, "
      f"TF-IDF: {tfidf_scores[i6] / tfidf_scores[i5]:.2f}x")
print("    'food' (in 1 document) is rarer than 'indian' (in 2 documents), so "
      "TF-IDF gives it more weight and D6 pulls further ahead of D5.")
print("\n===== END OF PRACTICAL =====")

```